Using facet for Generalizability Theory

Introduction to facet

The **facet** package provides comprehensive tools for conducting Generalizability Theory (G-theory) analyses using variance component models. It supports three estimation backends: frequentist (lme4), Bayesian (brms), and method of moments (mom/ANOVA-based). The package provides a unified interface for univariate and multivariate designs, with full support for G-studies, D-studies, and coefficient calculations with uncertainty quantification.

Note: The `tidy()` and `glance()` methods require the broom package to be loaded. The examples below use `library(broom)` to ensure these functions are available.

Overview

Generalizability Theory provides a framework for quantifying the reliability of measurements across multiple sources of error. The key concepts are:

- **G-studies**: Estimate variance components associated with different facets of measurement (e.g., items, raters, occasions)
- **D-studies**: Compute generalizability (G) and dependability (D/Phi) coefficients for different measurement designs

The facet package supports multiple backends:

- **lme4**: For frequentist estimation using linear mixed effects models (default for univariate models)
- **brms**: For Bayesian estimation using Stan (required for multivariate models)
- **mom**: For method of moments (ANOVA-based) estimation. Works best with balanced designs and provides confidence intervals using asymptotic approximations.

Installation

```r
# Install from GitHub
# install.packages("devtools")
devtools::install_github("yourname/facet")
```

Basic G-Study

A G-study estimates the variance components for each facet in your measurement design. The formula syntax follows lme4 conventions, specifying random effects with `(1 | facet)`.

```r
library(facet)
library(broom)
library(dplyr)
library(lme4)

# Create example data: 20 persons rated by 5 raters
set.seed(123)
data <- data.frame(
  score = rnorm(100, mean = 50, sd = 10),
```

```r
  person = factor(rep(1:20, 5)),
  rater = factor(rep(1:5, each = 20))
)

# Conduct G-study with default (lme4) backend
g <- gstudy(score ~ (1 | person) + (1 | rater), data = data)
#> boundary (singular) fit: see help('isSingular')

# View variance components
print(g)
#> Generalizability Study (G-Study)
#> ================================
#>
#> Backend: lme4
#> Formula: score ~ (1 | person) + (1 | rater)
#> Number of observations: 100
#> Multivariate: No
#>
#> Object of measurement: person
#> Facets: person, rater
#> Facet Sample Sizes:
#>   person: 20
#>   rater: 5
#>
#> Variance Components:
#> # A tibble: 3 x 4
#>   component facet    estimate   pct
#>   <chr>     <chr>       <dbl> <dbl>
#> 1 person    person        0       0
#> 2 rater     rater         0       0
#> 3 Residual  Residual     83.3   100
```

The output shows the estimated variance components for each facet:

- **person**: Variance attributable to differences between persons (universe score variance)
- **rater**: Variance attributable to differences between raters
- **Residual**: Remaining variance (person × rater interaction + error)

Viewing Results

Use `summary()` for a more detailed view, with options to display estimates on variance or SD scale:

```r
# Summary with variance estimates (default)
summary(g)
#> === G-Study Summary ===
#>
#> Design Information:
#>   Backend: lme4
#>   Formula: score ~ (1 | person) + (1 | rater)
#>   Observations: 100
#>   Multivariate: No
#>
#> Facet Information:
#>   Object of measurement: person
#>   Facets: person, rater
```

```
#>   Facet Sample Sizes:
#>     person: 20
#>     rater: 5
#>
#> Variance Components:
#>   person: 0.0000 (0.0%)
#>   rater: 0.0000 (0.0%)
#>   Residual: 83.3233 (100.0%)
#>
#> Total variance: 83.32328

# Summary with standard deviation estimates
summary(g, scale = "sd")
#> === G-Study Summary ===
#>
#> Design Information:
#>   Backend: lme4
#>   Formula: score ~ (1 | person) + (1 | rater)
#>   Observations: 100
#>   Multivariate: No
#>
#> Facet Information:
#>   Object of measurement: person
#>   Facets: person, rater
#>   Facet Sample Sizes:
#>     person: 20
#>     rater: 5
#>
#> Variance Components:
#>   person: 0.0000 (0.0%)
#>   rater: 0.0000 (0.0%)
#>   Residual: 9.1282 (100.0%)
#>
#> Total variance: 83.32328
```

Extract the variance components as a tibble using `tidy()`:

```
tidy(g)
#> # A tibble: 3 x 5
#>   component facet    type       var   pct
#>   <chr>     <chr>    <chr>    <dbl> <dbl>
#> 1 person    person   main       0       0
#> 2 rater     rater    main       0       0
#> 3 Residual  Residual residual  83.3   100
```

Use `glance()` for a one-row summary of the G-study:

```
glance(g)
#> # A tibble: 1 x 5
#>   n_obs n_facets backend is_multivariate object
#>   <int>    <int> <chr>   <lgl>           <chr>
#> 1   100        2 lme4    FALSE           person
```

G-Study with Confidence Intervals

For the lme4 backend, you can request confidence intervals using the `ci_method` parameter:

- `"none"` (default): No confidence intervals
- `"profile"`: Profile likelihood confidence intervals (more accurate, slower)
- `"boot"`: Parametric bootstrap confidence intervals (most accurate, slowest)

```
# G-study with profile likelihood confidence intervals
g_profile <- gstudy(
  score ~ (1 | person) + (1 | rater),
  data = data,
  ci_method = "profile"
)
#> boundary (singular) fit: see help('isSingular')
#> Computing profile confidence intervals ...

summary(g_profile)
#> === G-Study Summary ===
#>
#> Design Information:
#>   Backend: lme4
#>   Formula: score ~ (1 | person) + (1 | rater)
#>   Observations: 100
#>   Multivariate: No
#>
#> Facet Information:
#>   Object of measurement: person
#>   Facets: person, rater
#>   Facet Sample Sizes:
#>     person: 20
#>     rater: 5
#>
#> Variance Components:
#>   person: 0.0000 (0.0%)
#>   rater: 0.0000 (0.0%)
#>   Residual: 83.3233 (100.0%)
#>
#> Total variance: 83.32328
```

For bootstrap confidence intervals, you can specify the bootstrap type and number of simulations:

```
# G-study with bootstrap confidence intervals
# Using fewer iterations for speed in this example
g_boot <- gstudy(
  score ~ (1 | person) + (1 | rater),
  data = data,
  ci_method = "boot",
  nsim = 100,  # Number of bootstrap samples
  boot.type = "perc"  # Percentile bootstrap
)
#> boundary (singular) fit: see help('isSingular')
#> Computing bootstrap confidence intervals ...
#> ==========================================================================================
#>
#> 76 message(s): boundary (singular) fit: see help('isSingular')
```

```r
summary(g_boot)
#> === G-Study Summary ===
#>
#> Design Information:
#>   Backend: lme4
#>   Formula: score ~ (1 | person) + (1 | rater)
#>   Observations: 100
#>   Multivariate: No
#>
#> Facet Information:
#>   Object of measurement: person
#>   Facets: person, rater
#>   Facet Sample Sizes:
#>     person: 20
#>     rater: 5
#>
#> Variance Components:
#>   person: 0.0000 [0.0000, 14.4333] (0.0%)
#>   rater: 0.0000 [0.0000, 7.2695] (0.0%)
#>   Residual: 83.3233 [55.1136, 109.6346] (100.0%)
#>
#> Total variance: 83.32328
```

Method of Moments Backend

The method of moments (MOM) backend uses ANOVA-based variance component estimation. This approach works best with balanced designs and provides confidence intervals using asymptotic approximations.

```r
# Method of moments G-study
g_mom <- gstudy(
  score ~ (1 | person) + (1 | rater),
  data = data,
  backend = "mom"
)

print(g_mom)
#> Generalizability Study (G-Study)
#> ================================
#>
#> Backend: mom
#> Formula: score ~ (1 | person) + (1 | rater)
#> Number of observations: 100
#> Multivariate: No
#>
#> Object of measurement: person
#> Facets: person, rater
#> Facet Sample Sizes:
#>   person: 20
#>   rater: 5
#>
#> Variance Components:
#> # A tibble: 3 x 6
#>   component facet    estimate   pct `2.5% CI` `97.5% CI`
#>   <chr>     <chr>       <dbl> <dbl>     <dbl>      <dbl>
```

```
#> 1 person    person        0       0       0         46.7
#> 2 rater     rater         0       0       0        103.
#> 3 Residual  Residual     86.3   100      58.9      114.

# Summary shows confidence intervals
summary(g_mom)
#> === G-Study Summary ===
#>
#> Design Information:
#>   Backend: mom
#>   Formula: score ~ (1 | person) + (1 | rater)
#>   Observations: 100
#>   Multivariate: No
#>
#> Facet Information:
#>   Object of measurement: person
#>   Facets: person, rater
#>   Facet Sample Sizes:
#>     person: 20
#>     rater: 5
#>
#> Variance Components:
#>   person: 0.0000 [0.0000, 46.6529] (0.0%)
#>   rater: 0.0000 [0.0000, 102.7188] (0.0%)
#>   Residual: 86.2972 [58.8592, 113.7353] (100.0%)
#>
#> Total variance: 86.29725
```

The MOM backend automatically detects nested vs crossed designs:

```
# Create a crossed design data
set.seed(456)
crossed_data <- data.frame(
  score = rnorm(200, mean = 50, sd = 10),
  person = factor(rep(1:20, each = 10)),
  item = factor(rep(1:10, times = 20)),
  rater = factor(rep(1:5, each = 40))
)

g_crossed <- gstudy(
  score ~ (1 | person) + (1 | item) + (1 | rater),
  data = crossed_data,
  backend = "mom"
)
#> Warning in aov(as.formula(aov_formula), data = data): Error() model is singular

print(g_crossed)
#> Generalizability Study (G-Study)
#> ================================
#>
#> Backend: mom
#> Formula: score ~ (1 | person) + (1 | item) + (1 | rater)
#> Number of observations: 200
#> Multivariate: No
```

```
#>
#> Object of measurement: person
#> Facets: person, item, rater
#> Facet Sample Sizes:
#>   person: 20
#>   item: 10
#>   rater: 5
#>
#> Variance Components:
#> # A tibble: 3 x 6
#>   component facet    estimate   pct `2.5% CI` `97.5% CI`
#>   <chr>     <chr>       <dbl> <dbl>     <dbl>      <dbl>
#> 1 person    person       2.17  2.20       0         75.6
#> 2 item      item         2.60  2.63       0        137.
#> 3 Residual  Residual    93.9  95.2       74.0      114.
```

Bayesian Estimation with brms

For Bayesian estimation, use the brms backend. This provides:

- Posterior distributions for variance components
- Credible intervals (highest posterior density intervals)
- Full Bayesian inference including convergence diagnostics

```r
# Bayesian G-study with brms
# Note: Requires brms and Stan to be installed
g_bayes <- gstudy(
  score ~ (1 | person) + (1 | rater),
  data = data,
  backend = "brms"
)

print(g_bayes)
```

The brms backend provides additional diagnostic information in the summary:

```r
summary(g_bayes)
```

Bayesian Convergence Diagnostics

The summary includes Rhat (Gelman-Rubin diagnostic) and Effective Sample Size (ESS) values:

- **Rhat**: Should be close to 1.0 (indicates chain convergence)
- **Bulk_ESS**: Effective sample size for posterior mean estimates
- **Tail_ESS**: Effective sample size for posterior tail quantiles

D-Study

A D-study uses the G-study results to compute reliability coefficients for different measurement designs. The key coefficients are:

- **G coefficient**: Generalizability coefficient for *relative* decisions (comparing individuals' rankings)
- **D coefficient**: Dependability (Phi) coefficient for *absolute* decisions (comparing to a standard)

Basic D-Study

```
# Conduct D-study with 3 raters
d <- dstudy(g, n = list(rater = 3))

print(d)
#> Decision Study (D-Study)
#> ========================
#>
#> Based on G-Study with lme4 backend
#> Object of measurement: person
#> Error components for relative error (sigma2_delta): person:rater (Residual)
#> Error components for absolute error (sigma2_delta_abs): rater, person:rater (Residual)
#>
#> Sample Sizes:
#>  rater: 3
#>
#> Variance Components (scaled):
#> # A tibble: 3 x 4
#>   component facet    estimate   pct
#>   <chr>     <chr>       <dbl> <dbl>
#> 1 person    person        0       0
#> 2 rater     rater         0       0
#> 3 Residual  Residual     83.3   100
#>
#> Coefficients:
#> # A tibble: 1 x 7
#>   sigma2_p sigma2_delta sigma2_delta_abs g_coef d_coef sem_rel sem_abs
#>      <dbl>        <dbl>            <dbl>  <dbl>  <dbl>   <dbl>   <dbl>
#> 1        0         83.3             83.3      0      0    9.13    9.13
```

View detailed summary:

```
summary(d)
#> === D-Study Summary ===
#>
#> Object of measurement: person
#> Based on G-Study: score ~ (1 | person) + (1 | rater)
#>
#> Sample Sizes:
#>   rater: 3
#>
#> Variance Components (scaled):
#>   person: 0.0000 (0.0%)
#>   rater: 0.0000 (0.0%)
#>   Residual: 83.3233 (100.0%)
#>
#> Coefficients:
#> # A tibble: 1 x 7
#>   sigma2_p sigma2_delta sigma2_delta_abs g_coef d_coef sem_rel sem_abs
#>      <dbl>        <dbl>            <dbl>  <dbl>  <dbl>   <dbl>   <dbl>
#> 1        0         83.3             83.3      0      0    9.13    9.13
```

Exploring Different Sample Sizes

You can explore multiple sample size combinations using a sweep:

```
# Explore different numbers of raters
d_sweep <- dstudy(g, n = list(rater = c(1, 2, 3, 5, 10)))

# View all combinations
print(d_sweep)
#> Decision Study (D-Study)
#> ========================
#>
#> Based on G-Study with lme4 backend
#> Object of measurement: person
#> Error components for relative error (sigma2_delta): person:rater (Residual)
#> Error components for absolute error (sigma2_delta_abs): rater, person:rater (Residual)
#>
#> Sample Size Sweep:
#> # A tibble: 5 x 8
#>   rater sigma2_p sigma2_delta sigma2_delta_abs g_coef d_coef sem_rel sem_abs
#>   <dbl>    <dbl>        <dbl>            <dbl>  <dbl>  <dbl>   <dbl>   <dbl>
#> 1     1        0         83.3             83.3      0      0    9.13    9.13
#> 2     2        0         83.3             83.3      0      0    9.13    9.13
#> 3     3        0         83.3             83.3      0      0    9.13    9.13
#> 4     5        0         83.3             83.3      0      0    9.13    9.13
#> 5    10        0         83.3             83.3      0      0    9.13    9.13

# Coefficients for each design
tidy(d_sweep)
#> # A tibble: 5 x 8
#>   rater sigma2_p sigma2_delta sigma2_delta_abs g_coef d_coef sem_rel sem_abs
#>   <dbl>    <dbl>        <dbl>            <dbl>  <dbl>  <dbl>   <dbl>   <dbl>
#> 1     1        0         83.3             83.3      0      0    9.13    9.13
#> 2     2        0         83.3             83.3      0      0    9.13    9.13
#> 3     3        0         83.3             83.3      0      0    9.13    9.13
#> 4     5        0         83.3             83.3      0      0    9.13    9.13
#> 5    10        0         83.3             83.3      0      0    9.13    9.13
```

Specifying Object of Measurement

By default, the first facet in the formula is considered the object of measurement (typically "person"). You can specify this explicitly using the `object` parameter:

```
# Explicitly specify object of measurement
g_explicit <- gstudy(
  score ~ (1 | person) + (1 | rater),
  data = data,
  object = "person"
)
#> boundary (singular) fit: see help('isSingular')

# D-study with custom object specification
d_custom <- dstudy(g_explicit, n = list(rater = 3), object = "person")
```

The `object` parameter accepts multiple formats:

- Character string: `"person"`
- Character vector: `c("person", "person:rater")`
- Formula: `~ person + person:rater`

Advanced D-Study: Custom Error Specification

You can customize which variance components contribute to the error term using the `error` parameter:

```
# Create data with multiple facets
set.seed(789)
data_3facets <- data.frame(
  score = rnorm(200, mean = 50, sd = 10),
  person = factor(rep(1:20, each = 10)),
  item = factor(rep(1:10, times = 20)),
  rater = factor(rep(1:5, each = 40))
)

g_3f <- gstudy(
  score ~ (1 | person) + (1 | item) + (1 | rater),
  data = data_3facets
)
#> boundary (singular) fit: see help('isSingular')

print(g_3f)
#> Generalizability Study (G-Study)
#> ================================
#>
#> Backend: lme4
#> Formula: score ~ (1 | person) + (1 | item) + (1 | rater)
#> Number of observations: 200
#> Multivariate: No
#>
#> Object of measurement: person
#> Facets: person, item, rater
#> Facet Sample Sizes:
#>   person: 20
#>   item: 10
#>   rater: 5
#>
#> Variance Components:
#> # A tibble: 4 x 4
#>   component facet    estimate   pct
#>   <chr>     <chr>       <dbl> <dbl>
#> 1 person    person       0     0
#> 2 item      item         2.82  2.77
#> 3 rater     rater        1.45  1.42
#> 4 Residual  Residual    97.6  95.8

# D-study with custom error specification
d_error <- dstudy(
  g_3f,
  n = list(item = 10, rater = 3),
  error = c("person:item", "person:rater", "item:rater")
)
#> Warning: The residual component 'person:item' is already included in the model variance components. 
```

```r
summary(d_error)
#> === D-Study Summary ===
#>
#> Object of measurement: person
#> Based on G-Study: score ~ (1 | person) + (1 | item) + (1 | rater)
#>
#> Sample Sizes:
#>   item: 10
#>   rater: 3
#>
#> Variance Components (scaled):
#>   person: 0.0000 (0.0%)
#>   item: 2.8195 (2.8%)
#>   rater: 1.4497 (1.4%)
#>   Residual: 97.6461 (95.8%)
#>
#> Coefficients:
#> # A tibble: 1 x 7
#>   sigma2_p sigma2_delta sigma2_delta_abs g_coef d_coef sem_rel sem_abs
#>      <dbl>        <dbl>            <dbl>  <dbl>  <dbl>   <dbl>   <dbl>
#> 1        0            0                0    NaN    NaN       0       0
```

The `error` parameter accepts: - Character string: `"person:item"` - Character vector: `c("person:item", "person:rater")` - Formula: `~ person:item + person:rater`

D-Study with Aggregation

When aggregating over a facet (e.g., taking the mean across multiple raters), use the `aggregation` parameter to correctly scale the variance components:

```r
# D-study with aggregation over raters
# This models the case where we average scores across raters
d_agg <- dstudy(
  g_3f,
  n = list(item = 10, rater = 3),
  object = "person",
  aggregation = "rater",
  residual_is = "person:item:rater"
)

summary(d_agg)
#> === D-Study Summary ===
#>
#> Object of measurement: person
#> Based on G-Study: score ~ (1 | person) + (1 | item) + (1 | rater)
#>
#> Sample Sizes:
#>   item: 10
#>   rater: 3
#>
#> Variance Components (scaled):
#>   person: 0.0000 (0.0%)
#>   item: 0.2820 (0.8%)
#>   rater: 0.4832 (1.5%)
```

```
#>   Residual: 32.5487 (97.7%)
#>
#> Coefficients:
#> # A tibble: 1 x 7
#>   sigma2_p sigma2_delta sigma2_delta_abs g_coef d_coef sem_rel sem_abs
#>      <dbl>        <dbl>            <dbl>  <dbl>  <dbl>   <dbl>   <dbl>
#> 1        0         32.5             32.8      0      0    5.71    5.73
```

The `residual_is` parameter specifies which facets make up the residual variance component (important for proper scaling when aggregating).

Posterior-Based D-Study Coefficients

For Bayesian G-studies, you can compute D-study coefficients using the full posterior distributions:

```r
# D-study with posterior estimation (requires brms backend)
# Note: This uses full posterior to compute coefficients with uncertainty
d_post <- dstudy(
  g_bayes,
  n = list(rater = 3),
  estimation = "posterior"
)

print(d_post)

# Access posterior distributions
names(d_post$posterior)
```

If you request posterior estimation but the G-study was not fit with brms, the function will automatically refit:

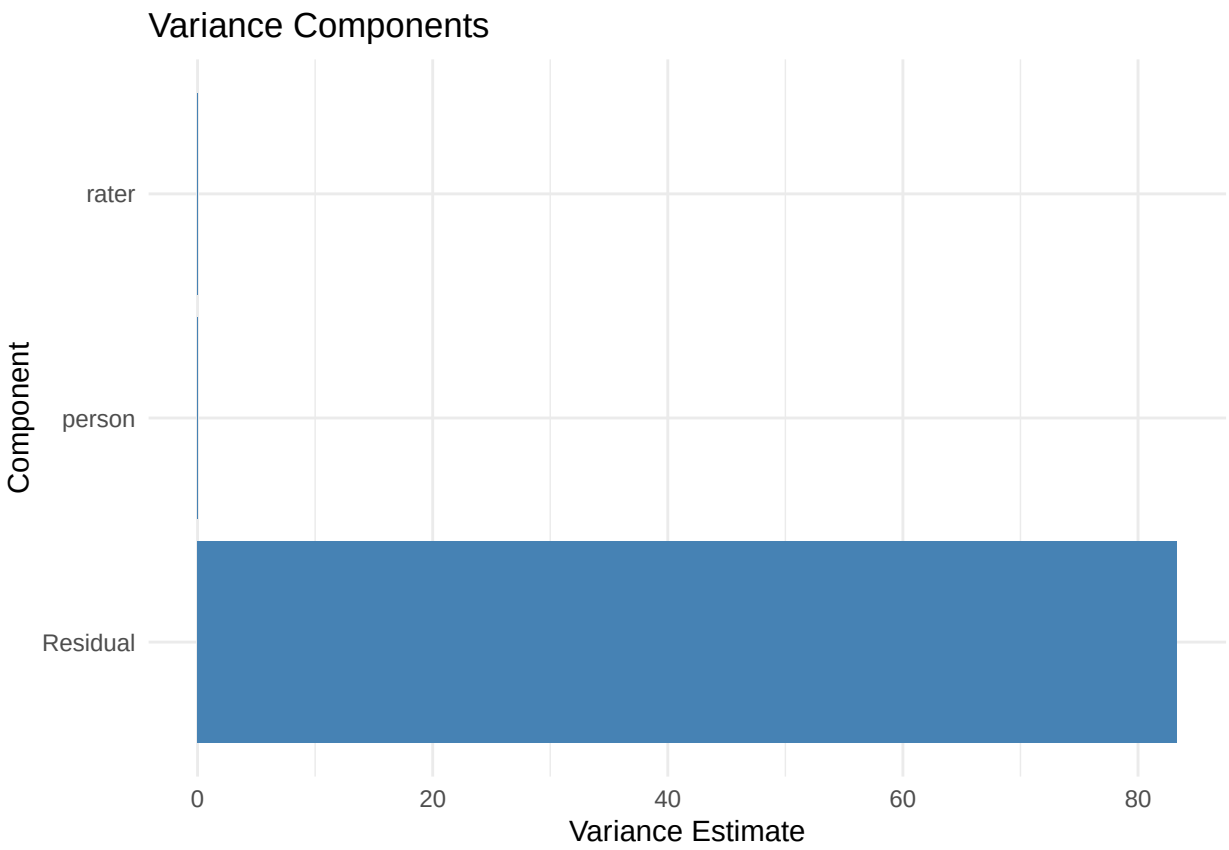
```r
# D-study with posterior from lme4 gstudy
d_post2 <- dstudy(
  g,
  n = list(rater = 3),
  estimation = "posterior"
)
```

Visualization

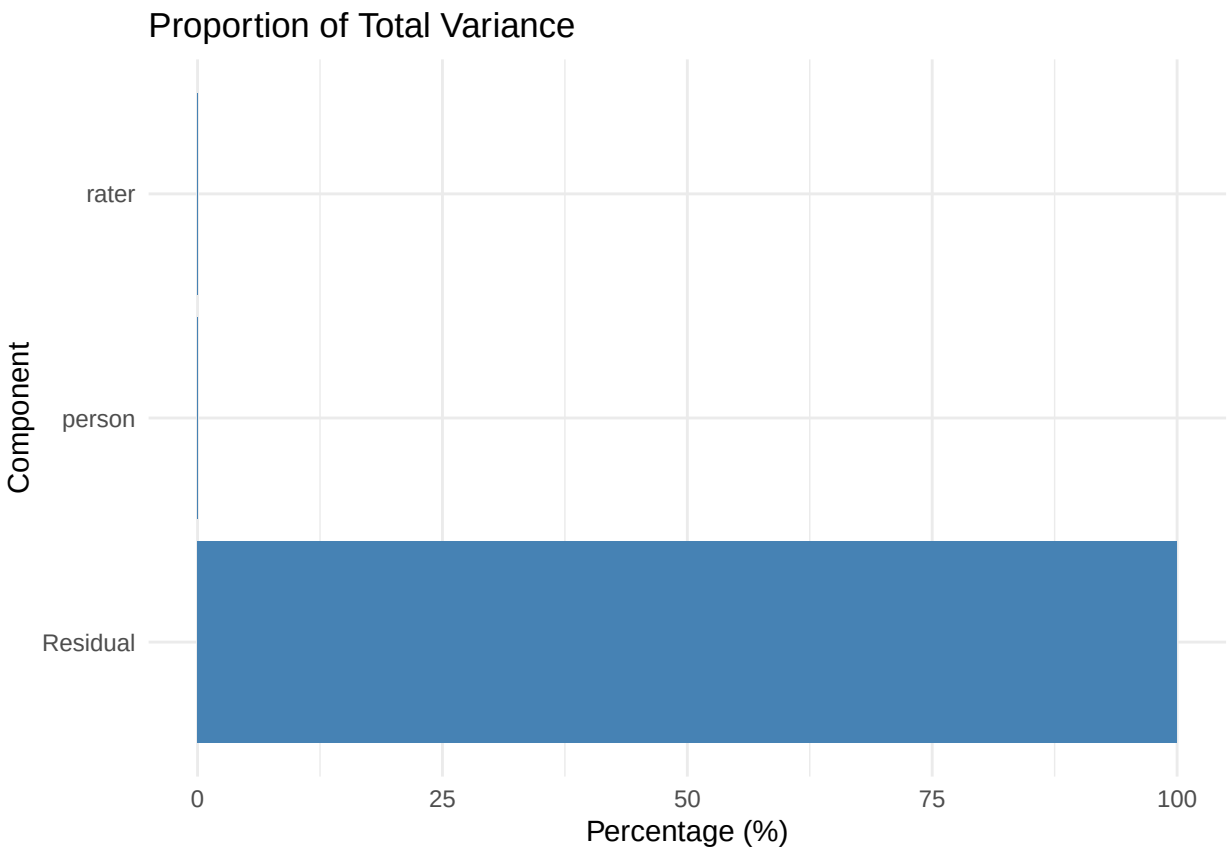
G-Study Plots

Plot variance components with different plot types:

```r
# Bar plot of variance components
plot(g, type = "variance")
```




```r
# Proportion plot
plot(g, type = "proportion")
```

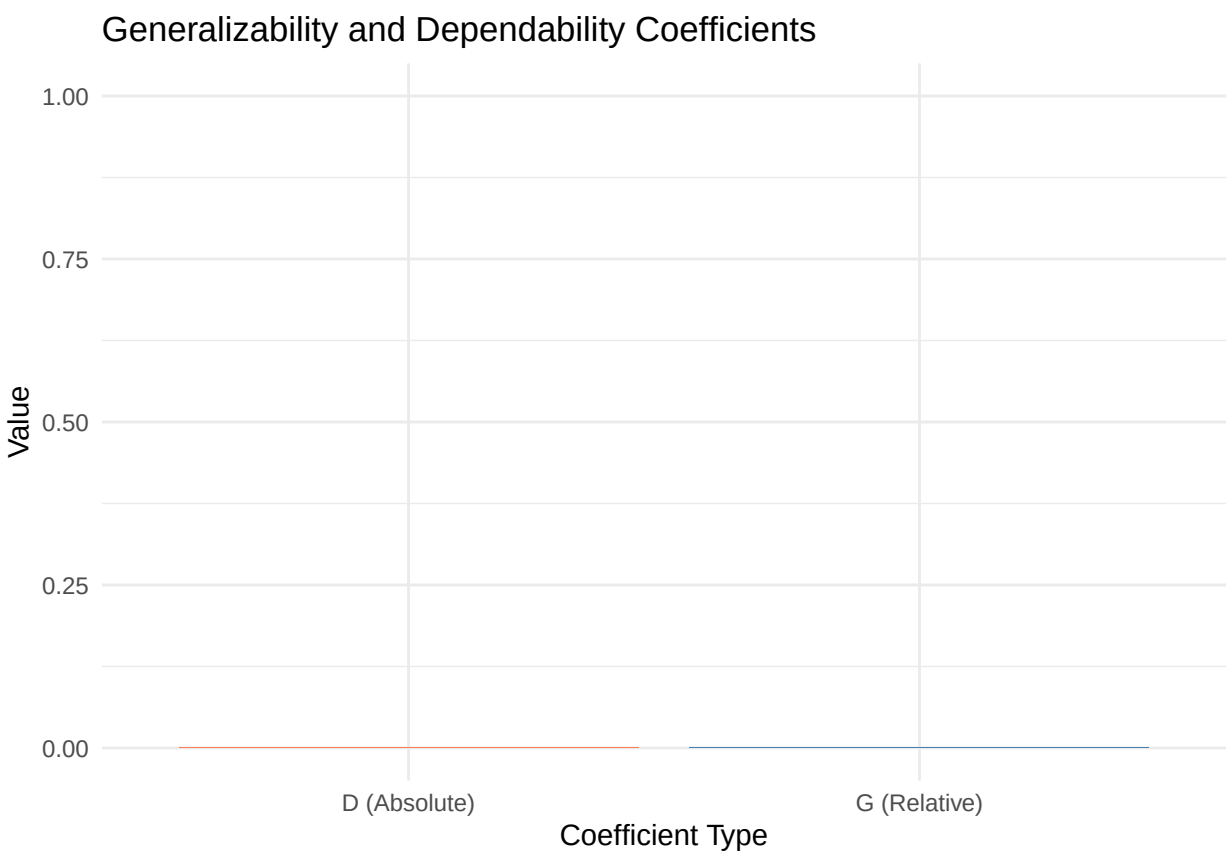



```
# Forest plot with confidence intervals (for brms or with ci_method)
plot(g_bayes, type = "forest")
```

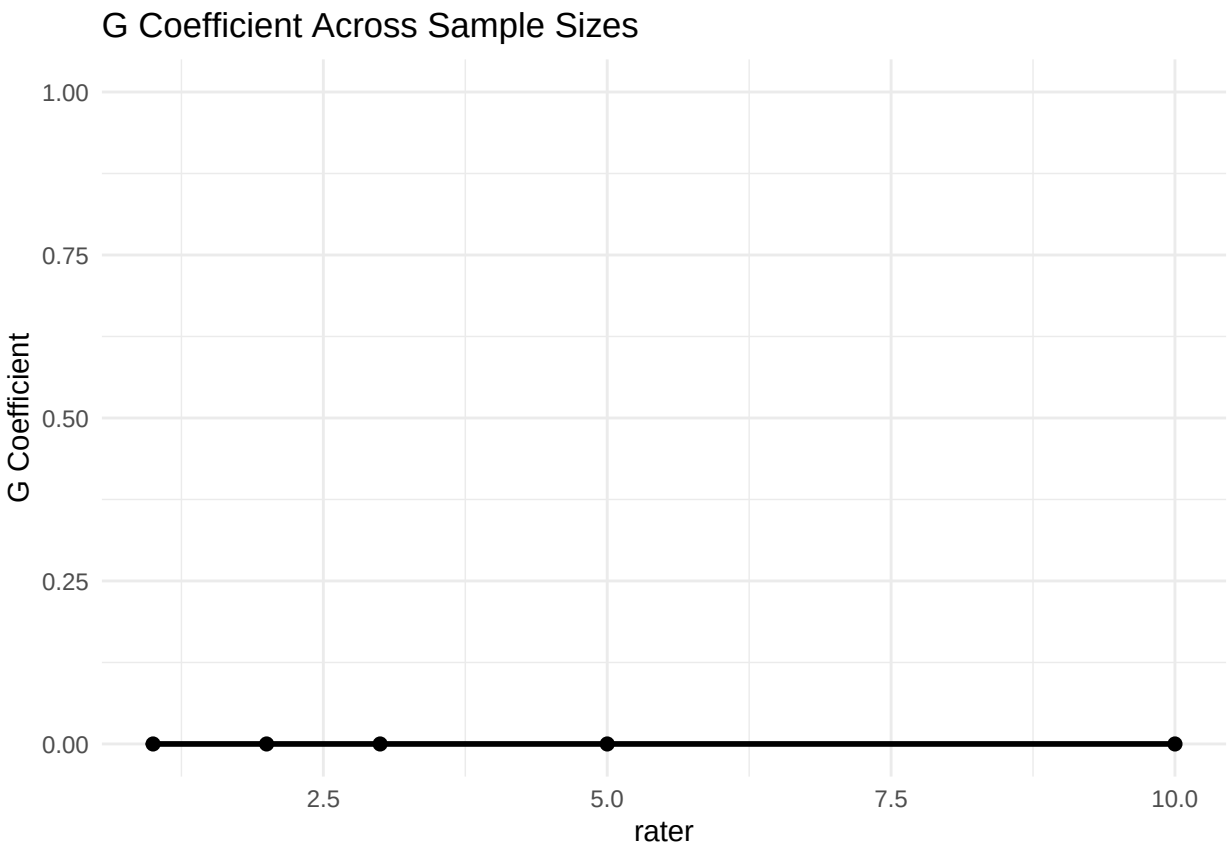
D-Study Plots

Plot D-study coefficients:

```
# Bar plot of G and D coefficients
plot(d, type = "coefficients")
```



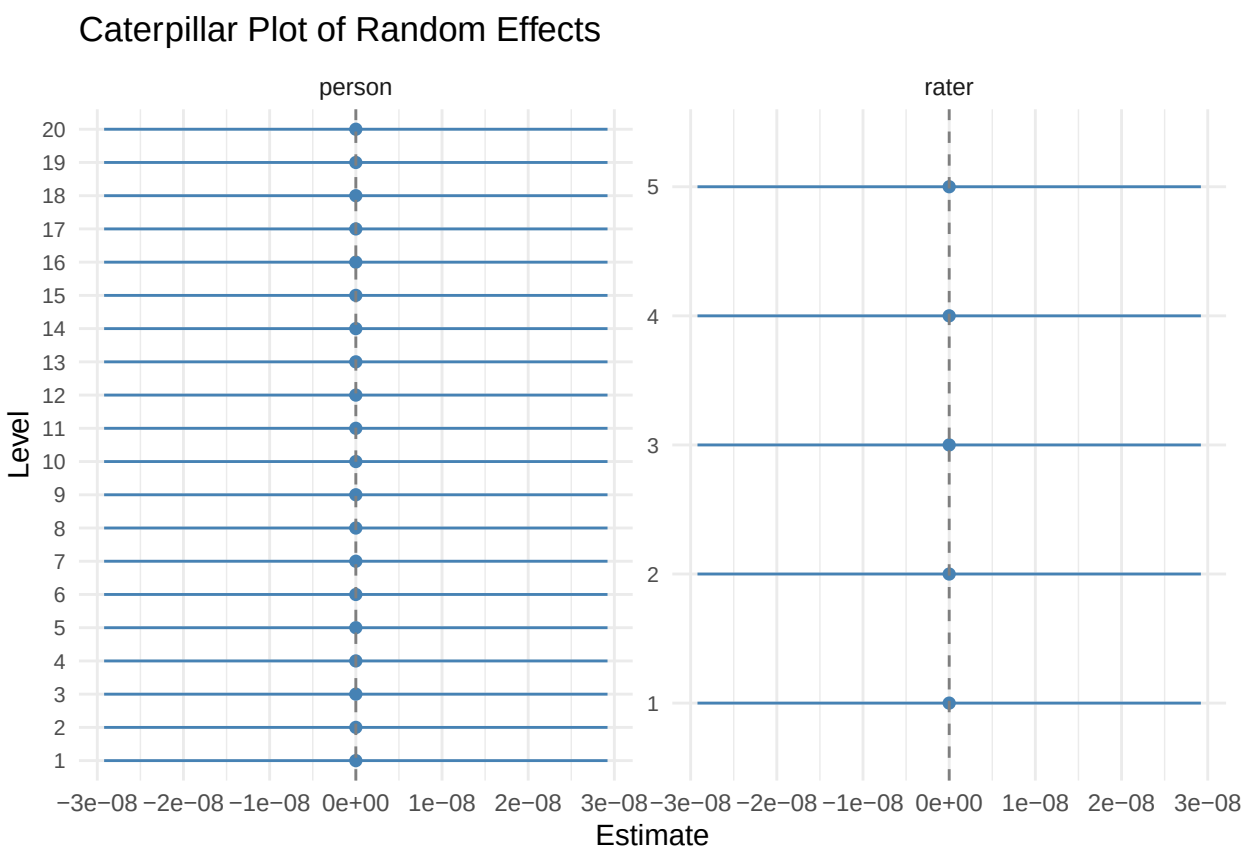

```r
# Sweep plot (for sample size exploration)
plot(d_sweep, type = "sweep")
```



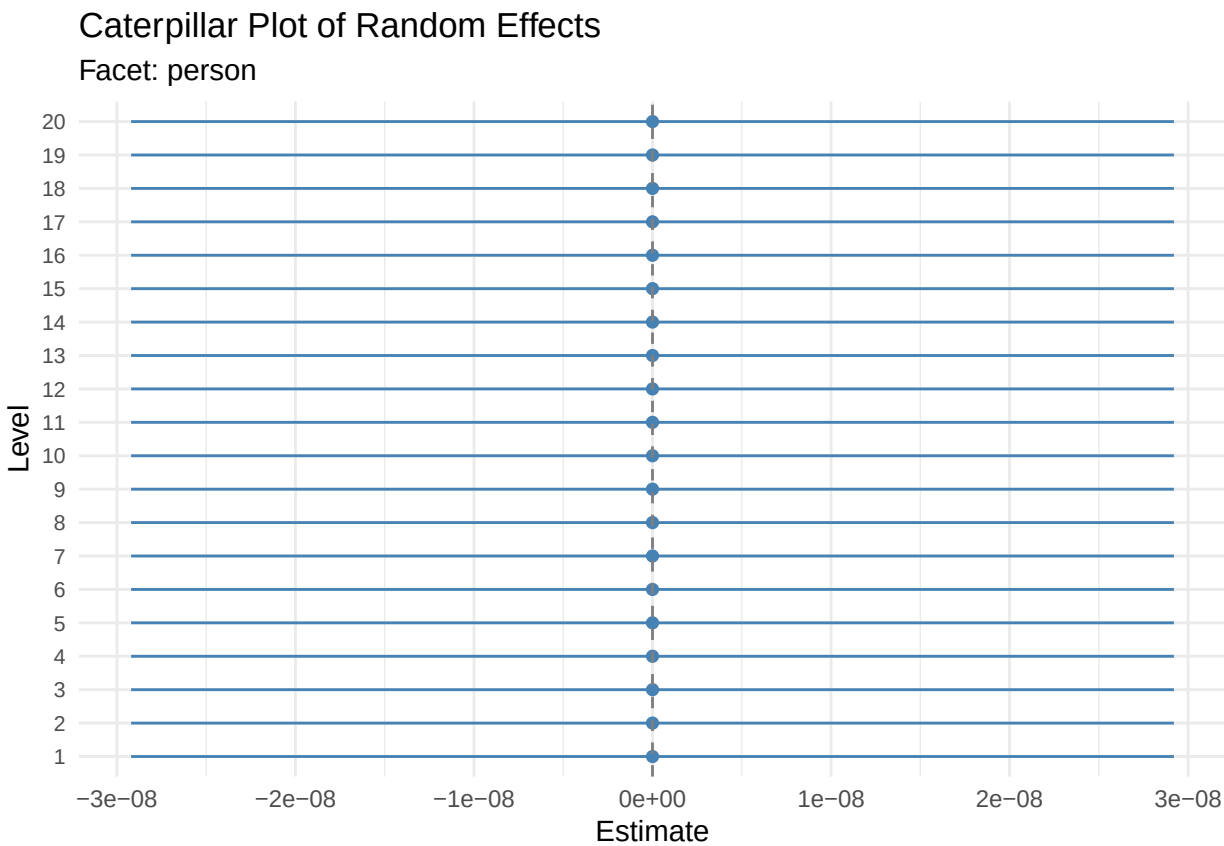
Caterpillar Plots for Random Effects

Create caterpillar (dot plot with error bars) to display random effects:

```
# Plot all random effects
plot_ranef(g)
```




```
# Plot specific random effect
plot_ranef(g, which = "person")
```



For Bayesian models, caterpillar plots show credible intervals:

```
# Plot with brms backend
plot_ranef(g_bayes, which = "rater")
```

Working with Results

Extract Variance Components

```
# Get variance components as data frame
vc <- tidy(g)
vc
#> # A tibble: 3 x 5
#>   component facet    type       var   pct
#>   <chr>     <chr>    <chr>    <dbl> <dbl>
#> 1 person    person   main       0       0
#> 2 rater     rater    main       0       0
#> 3 Residual  Residual residual  83.3   100
```

Filter and Manipulate

```
library(dplyr)

# Filter specific components
vc %>%
  filter(component != "Residual") %>%
  mutate(pct_total = var / sum(var) * 100)
```

```
#> # A tibble: 2 x 6
#>   component facet  type    var   pct pct_total
#>   <chr>     <chr>  <chr> <dbl> <dbl>     <dbl>
#> 1 person    person main      0     0       NaN
#> 2 rater     rater  main      0     0       NaN
```

Access Model Objects

The underlying model object can be accessed from a gstudy object:

```
# Access the fitted model
model <- g$model
print(summary(model))
#> Linear mixed model fit by REML ['lmerMod']
#> Formula: score ~ (1 | person) + (1 | rater)
#>    Data: data
#>
#> REML criterion at convergence: 723.4
#>
#> Scaled residuals:
#>      Min       1Q   Median       3Q      Max
#> -2.62876 -0.64006 -0.03139  0.65885  2.29721
#>
#> Random effects:
#>  Groups   Name        Variance Std.Dev.
#>  person   (Intercept)  0.00    0.000
#>  rater    (Intercept)  0.00    0.000
#>  Residual             83.32    9.128
#> Number of obs: 100, groups:  person, 20; rater, 5
#>
#> Fixed effects:
#>             Estimate Std. Error t value
#> (Intercept)  50.9041     0.9128   55.77
#> optimizer (nloptwrap) convergence code: 0 (OK)
#> boundary (singular) fit: see help('isSingular')
```

Note: The `VarCorr()` and `ranef()` functions work with the underlying model when lme4 is loaded:

```
# Extract variance-covariance matrices (requires loading lme4)
library(lme4)
vc_obj <- VarCorr(g)
print(vc_obj)

# Extract random effects (BLUPs/conditional modes)
re <- ranef(g)
print(re)
```

For brms models:

```
# Extract random effects from Bayesian model
re_bayes <- ranef(g_bayes)
print(re_bayes)

# Parameter pairs plot
# pairs(g_bayes)
```

Extract Posterior Draws

For brms backend, extract posterior draws for custom analyses:

```
# Extract posterior draws
# draws <- extract_draws(g_bayes)
```

Multivariate Analysis

For multivariate designs (multiple outcome variables), use brms syntax with `mvbind()`:

```
# Create multivariate data
set.seed(101)
multi_data <- data.frame(
  score1 = rnorm(100, mean = 50, sd = 10),
  score2 = rnorm(100, mean = 30, sd = 5),
  person = factor(rep(1:20, 5)),
  rater = factor(rep(1:5, each = 20))
)

# Multivariate G-study (automatically uses brms)
g_mv <- gstudy(
  mvbind(score1, score2) ~ (1 | person) + (1 | rater),
  data = multi_data
)

print(g_mv)
```

View multivariate results:

```
summary(g_mv)
```

D-study for multivariate models:

```
d_mv <- dstudy(g_mv, n = list(rater = 3))
print(d_mv)
```

Three-Facet Designs

A common design includes person, item, and rater facets:

```
# Create three-facet data
set.seed(202)
data_3f <- data.frame(
  score = rnorm(300, mean = 50, sd = 10),
  person = factor(rep(1:20, each = 15)),
  item = factor(rep(1:15, times = 20)),
  rater = factor(rep(1:3, each = 100))
)

# G-study with three facets
g_3facet <- gstudy(
  score ~ (1 | person) + (1 | item) + (1 | rater),
  data = data_3f
)
#> boundary (singular) fit: see help('isSingular')
```

```r
print(g_3facet)
#> Generalizability Study (G-Study)
#> ================================
#>
#> Backend: lme4
#> Formula: score ~ (1 | person) + (1 | item) + (1 | rater)
#> Number of observations: 300
#> Multivariate: No
#>
#> Object of measurement: person
#> Facets: person, item, rater
#> Facet Sample Sizes:
#>   person: 20
#>   item: 15
#>   rater: 3
#>
#> Variance Components:
#> # A tibble: 4 x 4
#>   component facet    estimate   pct
#>   <chr>     <chr>       <dbl> <dbl>
#> 1 person    person       2.05  1.94
#> 2 item      item         0     0
#> 3 rater     rater        0     0
#> 4 Residual  Residual   104.   98.1

# D-study: what if we used 5 items rated by 2 raters?
d_3f <- dstudy(
  g_3facet,
  n = list(item = 5, rater = 2)
)

summary(d_3f)
#> === D-Study Summary ===
#>
#> Object of measurement: person
#> Based on G-Study: score ~ (1 | person) + (1 | item) + (1 | rater)
#>
#> Sample Sizes:
#>   item: 5
#>   rater: 2
#>
#> Variance Components (scaled):
#>   person: 2.0540 (1.9%)
#>   item: 0.0000 (0.0%)
#>   rater: 0.0000 (0.0%)
#>   Residual: 103.9054 (98.1%)
#>
#> Coefficients:
#> # A tibble: 1 x 7
#>   sigma2_p sigma2_delta sigma2_delta_abs g_coef d_coef sem_rel sem_abs
#>      <dbl>        <dbl>            <dbl>  <dbl>  <dbl>   <dbl>   <dbl>
#> 1     2.05         104.             104. 0.0194 0.0194    10.2    10.2
```

Complete Example

Here's a complete workflow demonstrating multiple features:

```r
library(facet)
library(dplyr)

# 1. Prepare data
set.seed(303)
data <- data.frame(
  score = rnorm(400, mean = 50, sd = 10),
  person = factor(rep(1:40, each = 10)),
  item = factor(rep(1:10, times = 40)),
  rater = factor(rep(1:2, each = 200))
)

# 2. Conduct G-study with profile confidence intervals
g <- gstudy(
  score ~ (1 | person) + (1 | item) + (1 | rater),
  data = data,
  ci_method = "profile"
)
#> boundary (singular) fit: see help('isSingular')
#> Computing profile confidence intervals ...

# 3. View variance components
print(g)
#> Generalizability Study (G-Study)
#> ================================
#>
#> Backend: lme4
#> Formula: score ~ (1 | person) + (1 | item) + (1 | rater)
#> Number of observations: 400
#> Multivariate: No
#>
#> Object of measurement: person
#> Facets: person, item, rater
#> Facet Sample Sizes:
#>   person: 40
#>   item: 10
#>   rater: 2
#>
#> Variance Components:
#> # A tibble: 4 x 6
#>   component facet    estimate    pct `2.5% CI` `97.5% CI`
#>   <chr>     <chr>       <dbl>  <dbl>     <dbl>      <dbl>
#> 1 person    person      0.505  0.508       0         6.49
#> 2 item      item        0      0           0         1.32
#> 3 rater     rater       0      0           0         3.31
#> 4 Residual  Residual   98.9   99.5        85.8     114.

# 4. Summary with confidence intervals
summary(g)
#> === G-Study Summary ===
#>
```

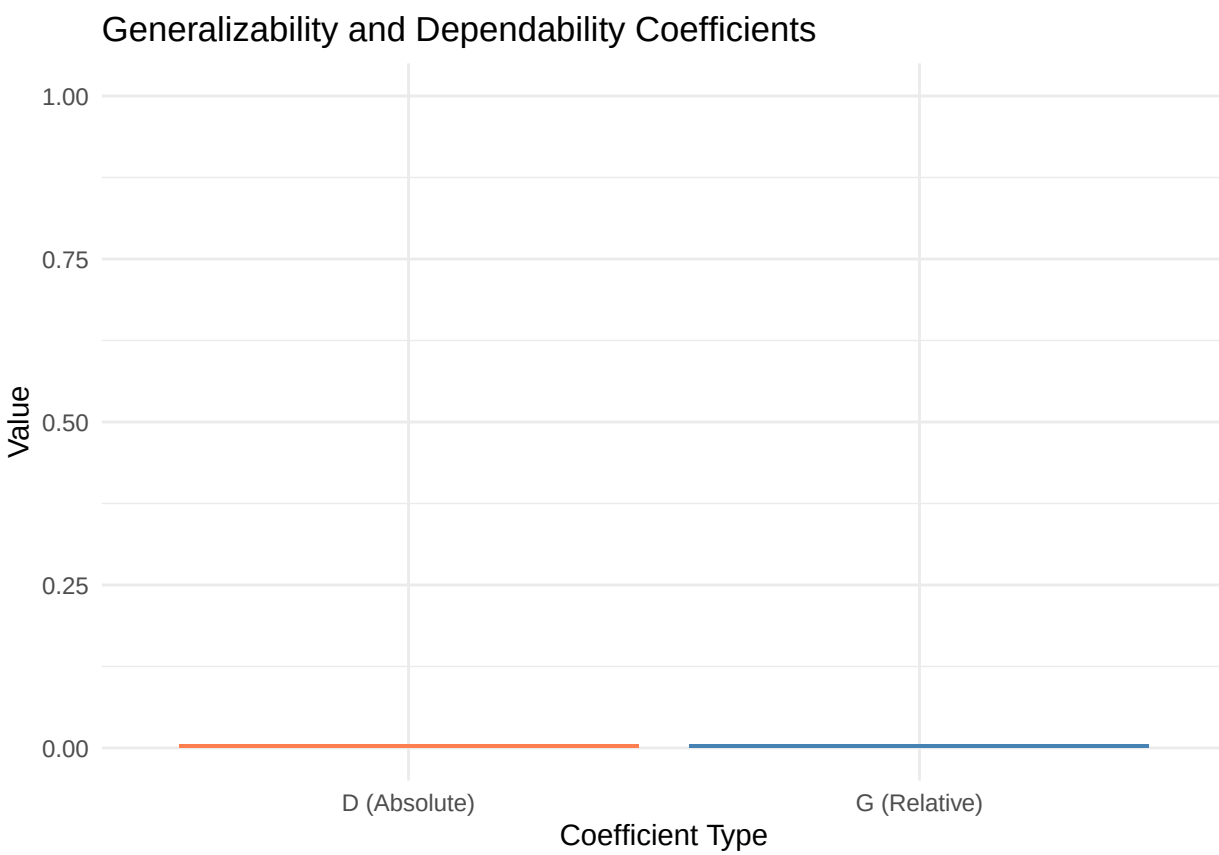
```
#> Design Information:
#>   Backend: lme4
#>   Formula: score ~ (1 | person) + (1 | item) + (1 | rater)
#>   Observations: 400
#>   Multivariate: No
#>
#> Facet Information:
#>   Object of measurement: person
#>   Facets: person, item, rater
#>   Facet Sample Sizes:
#>     person: 40
#>     item: 10
#>     rater: 2
#>
#> Variance Components:
#>   person: 0.5053 [0.0000, 6.4912] (0.5%)
#>   item: 0.0000 [0.0000, 1.3171] (0.0%)
#>   rater: 0.0000 [0.0000, 3.3120] (0.0%)
#>   Residual: 98.8679 [85.7566, 114.1971] (99.5%)
#>
#> Total variance: 99.37312

# 5. Conduct D-study with different designs
d <- dstudy(g, n = list(item = 10, rater = 2))

# 6. View coefficients
summary(d)
#> === D-Study Summary ===
#>
#> Object of measurement: person
#> Based on G-Study: score ~ (1 | person) + (1 | item) + (1 | rater)
#>
#> Sample Sizes:
#>   item: 10
#>   rater: 2
#>
#> Variance Components (scaled):
#>   person: 0.5053 [0.0000, 6.4912] (0.5%)
#>   item: 0.0000 [0.0000, 1.3171] (0.0%)
#>   rater: 0.0000 [0.0000, 3.3120] (0.0%)
#>   Residual: 98.8679 [85.7566, 114.1971] (99.5%)
#>
#> Coefficients:
#> # A tibble: 1 x 7
#>   sigma2_p sigma2_delta sigma2_delta_abs  g_coef  d_coef sem_rel sem_abs
#>      <dbl>        <dbl>            <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
#> 1    0.505         98.9             98.9 0.00508 0.00508    9.94    9.94

# 7. Explore alternative designs
d_alt <- dstudy(g, n = list(item = c(5, 10, 15), rater = c(1, 2, 3)))

# 8. Plot sweep results
plot(d_alt)
```




```
# 9. Filter and analyze variance components
tidy(g) %>%
  filter(component != "Residual") %>%
  arrange(desc(var)) %>%
  mutate(cum_pct = cumsum(pct))
#> # A tibble: 3 x 8
#>   component facet  type    var   pct lower upper cum_pct
#>   <chr>     <chr>  <chr> <dbl> <dbl> <dbl> <dbl>   <dbl>
#> 1 person    person main  0.505 0.508     0  6.49   0.508
#> 2 item      item   main  0     0         0  1.32   0.508
#> 3 rater     rater  main  0     0         0  3.31   0.508
```

Utility Functions

Predicates

Check object types:

```
is.gstudy(g)
#> [1] TRUE
is.dstudy(d)
#> [1] TRUE
```

Scale Parameter

Control display scale in print and summary methods:

```r
# Display on SD scale
print(g, scale = "sd")
#> Generalizability Study (G-Study)
#> ================================
#>
#> Backend: lme4
#> Formula: score ~ (1 | person) + (1 | item) + (1 | rater)
#> Number of observations: 400
#> Multivariate: No
#>
#> Object of measurement: person
#> Facets: person, item, rater
#> Facet Sample Sizes:
#>   person: 40
#>   item: 10
#>   rater: 2
#>
#> Variance Components:
#> # A tibble: 4 x 6
#>   component facet    estimate    pct `2.5% CI` `97.5% CI`
#>   <chr>     <chr>       <dbl>  <dbl>     <dbl>      <dbl>
#> 1 person    person      0.711  0.508      0          2.55
#> 2 item      item        0      0          0          1.15
#> 3 rater     rater       0      0          0          1.82
#> 4 Residual  Residual    9.94  99.5        9.26      10.7
summary(g, scale = "sd")
#> === G-Study Summary ===
#>
#> Design Information:
#>   Backend: lme4
#>   Formula: score ~ (1 | person) + (1 | item) + (1 | rater)
#>   Observations: 400
#>   Multivariate: No
#>
#> Facet Information:
#>   Object of measurement: person
#>   Facets: person, item, rater
#>   Facet Sample Sizes:
#>     person: 40
#>     item: 10
#>     rater: 2
#>
#> Variance Components:
#>   person: 0.7108 [0.0000, 2.5478] (0.5%)
#>   item: 0.0000 [0.0000, 1.1477] (0.0%)
#>   rater: 0.0000 [0.0000, 1.8199] (0.0%)
#>   Residual: 9.9432 [9.2605, 10.6863] (99.5%)
#>
#> Total variance: 99.37312
```

References

Brennan, R. L. (2001). *Generalizability theory*. Springer.

Shavelson, R. J., & Webb, N. M. (1991). *Generalizability theory: A primer*. Sage Publications.